Nom:

Prénom:

BACCALAURÉAT GÉNÉRAL

ÉPREUVE D'ENSEIGNEMENT DE SPÉCIALITÉ

Session 2026

NUMERIQUE ET SCIENCES INFORMATIQUES

DURÉE DE L'ÉPREUVE : 3 heures 30

Ce sujet comporte 19 pages numérotées de 1 à 19
La dernière page comporte une annexe que vous pouvez détacher.
Le candidat doit traiter les trois exercices proposés.

L'utilisation d'une calculatrice n'est pas autorisée

CORRECTION

Exercice 1 ▶ /7 points

PARTIE A

Un coeur de réseau est constitué d'un maillage dans lequel les routeurs R_1 à R_6 sont reliés par des liaisons physiques dont le débit en Mbps (Méga-bits par seconde) est indiqué sur la figure suivante.

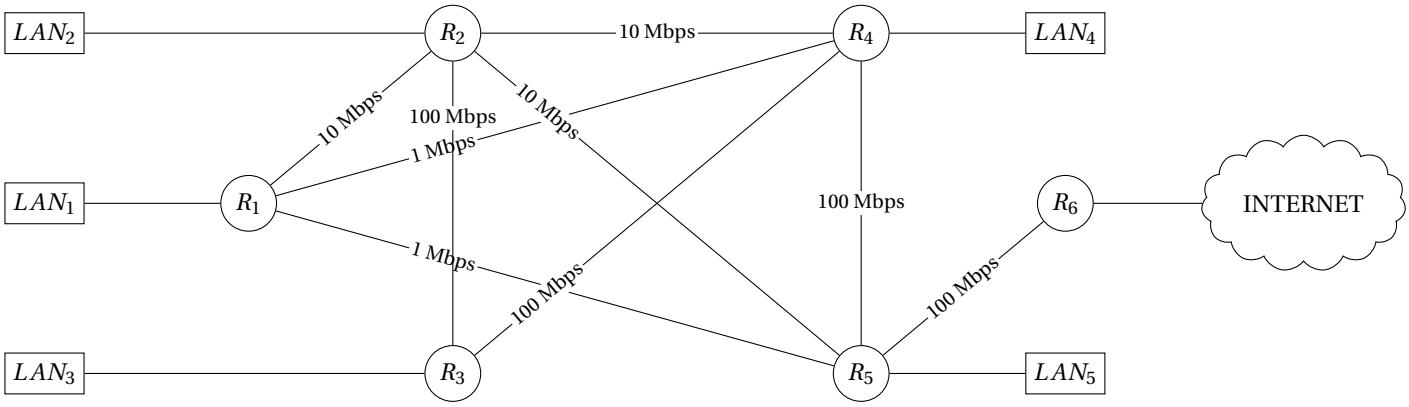


FIGURE 1 – Schéma du réseau

Derrière chaque routeur R_1 à R_5, un switch distribue un réseau local LAN_1 à LAN_5. Les protocoles utilisés sont le protocole RIP et le protocole OSPF, qui minimisent respectivement le nombre de routeurs traversés et la somme des coûts des liaisons physiques empruntées.

1. Compléter la table de routage de R_1 sachant que le protocole de routage RIP est utilisé. (/0.75)
   La destination est le routeur à atteindre, le prochain saut est le premier routeur traversé et la distance est le nombre de routeurs traversés.
2. Un utilisateur du réseau local LAN_1 interroge, via son navigateur web, un serveur d'Internet. Détailler, suivant le protocole RIP, la route suivie dans le maillage par la requête à destination de ce serveur. (/0.5)

$R_1 \rightarrow R_5 \rightarrow R_6 \rightarrow INTERNET.$

Table de routage R_1		
destination	prochain saut	distance
R_2	R_2	0
R_3	R_2	1
R_4	R_4	0
R_5	R_5	0
R_6	R_5	1

Le coût d'une liaison est donné par la relation coût $\dfrac{10^2}{\text{débit}}$ où le débit est le débit de la liaison en Mbps.

Coût des liaisons depuis R_1			
routeur	R_2	R_4	R_5
coût	10	100	100

3. Compléter la table de routage de R_1 sachant que le protocole de routage OSPF est utilisé. (/0.75)
   Un utilisateur du réseau local LAN_1 interroge, via son navigateur web, un serveur d'Internet.
4. Donner, selon le protocole OSPF, la route suivie dans le maillage par la requête à destination de ce serveur. (/0.5)

$R_1 \rightarrow R_2 \rightarrow R_3 \rightarrow R_4 \rightarrow R_5 \rightarrow R_6 \rightarrow INTERNET.$

Table de routage R_1		
destination	prochain saut	distance
R_2	R_2	10
R_3	R_2	11
R_4	R_2	12
R_5	R_2	13
R_6	R_2	14

Le protocole de routage OSPF est toujours utilisé et le routeur R_2 tombe en panne.

5. Donner la nouvelle route suivie dans le maillage par une requête partant du réseau LAN_1 à destination d'Internet et en donner la nouvelle valeur de la distance. (/0.5)

$R_1 \rightarrow R_5 \rightarrow R_6 \rightarrow INTERNET$ et la distance est de 101.

PARTIE B

Le réseau LAN_1 est supposé disposer d'un serveur DHCP (Dynamic Host Control Protocol) gérant l'attribution d'adresses IPv4.

Il est rappelé, ici, qu'une adresse IPv4 est une séquence de 4 octets, généralement représentée par les 4 entiers correspondants en écriture décimale, séparés par des points (notation décimale pointée).

Un réseau est caractérisé par des machines dont les adresses ont en commun les n mêmes premiers bits.

La notation CIDR du réseau a la forme "adresse réseau / n". Appliqué à une adresse du réseau, le masque permet de calculer le préfixe réseau commun. Il est constitué de 4 octets consécutifs, dont (de gauche à droite) les n premiers bits sont égaux à 1 et les suivants à 0. On obtient l'adresse du réseau en effectuant un ET logique, bit à bit, entre chaque octet composant l'adresse d'une machine et l'octet qui lui correspond dans le masque.

Exemple de `ET` :

```
    l'entier 192 s'écrit : 1 1 0 0 0 0 0 0
    l'entier 255 s'écrit : 1 1 1 1 1 1 1 1
-------------------------------------------------------
   ET logique, bit à bit : 1 1 0 0 0 0 0 0 ( soit l'entier 192 )
```

6. Compléter le tableau suivant correspondant à une machine d'un réseau d'adresse `192.168.1.100` et de masque `255.192.0.0`. Ce tableau détermine l'adresse du réseau (en binaire puis en décimale pointée). (/0.5)

Calcul d'une adresse de réseau				
machine (binaire)	11000000	10101000	00000001	01100100
masque (binaire)	11111111	11000000	00000000	00000000
réseau (binaire)	11000000	10000000	00000000	00000000
réseau (déc. pointée)	192	128	0	0

On obtient le complémentaire d'un octet en échangeant respectivement chacun des 0 et des 1 qui le composent, par un 1 ou un 0.

Par exemple, le complémentaire de `1 1 0 0 1 0 1 0` est `0 0 1 1 0 1 0 1`.

Par un procédé analogue à celui de la question précédente, l'adresse de broadcast d'un réseau est obtenue en effectuant un OU logique bit à bit entre chaque octet composant l'adresse réseau et le complémentaire de l'octet correspondant dans l'écriture binaire du masque.

Exemple de `OU` :

```
           12 s'écrit : 0 0 0 0 1 1 0 0
            9 s'écrit : 0 0 0 0 1 0 0 1
-------------------------------------------------------
OU logique, bit à bit : 0 0 0 0 1 1 0 1 ( soit l'entier 13 )
```

7. Compléter le tableau ci-après pour déterminer l'adresse de broadcast du réseau suivant (en binaire puis en décimale pointée). (/0.5)

Calcul d'une adresse de broadcast				
réseau (binaire)	11000000	10000000	00000000	00000000
masque (binaire)	11111111	11000000	00000000	00000000
complément du masque (binaire)	00000000	00111111	11111111	11111111
broadcast (binaire)	11000000	10111111	11111111	11111111
broadcast (déc. pointée)	192	191	255	255

Une machine du réseau LAN_1 a reçu du serveur DHCP l'adresse `172.16.1.100`, avec le masque `255.255.0.0`.

Sa passerelle par défaut a pour adresse `172.16.255.254`.

8. En déduire les informations suivantes : (/1)
   - l'adresse du réseau LAN_1 (en décimale pointée) ;

     L'adresse du réseau LAN_1 est `172.16.0.0`.
   - l'adresse de broadcast (en décimale pointée) ;

     L'adresse de broadcast du réseau LAN_1 est `172.16.255.255`.
   - le nombre total d'adresses pouvant être distribuées par le serveur, en ne tenant pas compte des éventuelles restrictions possiblement posées par l'administrateur du réseau.

     Le nombre total d'adresses pouvant être distribuées par le serveur DHCP est de $2^{16} - 2 = 65534$ adresses, car il faut exclure l'adresse du réseau et l'adresse de broadcast.

On souhaite, désormais, simuler en Python le fonctionnement du réseau LAN_1.

On donne, ci-après, les documentations des méthodes `split` et `join`.

- la documentation de la méthode `split` de l'objet `str` est la suivante :

Code Python
```python
split(self, séparateur)
    Renvoie la liste des sous-chaines de caractères délimitées par
    le séparateur dans l'instance courante de chaîne de caractères.
    >>> 'ab-pq-rs'.split('-')
    ['ab', 'pq', 'rs']
```

- la documentation de la méthode `join` de l'objet `str` :

Code Python
```python
join(self, iterable)
    Fusionne les chaînes de caractères contenues dans iterable en
    le liant par la sous-chaîne depuis laquelle cette méthode est appelée.
    >>>'.'.join(['ab', 'pq', 'rs'])
    'ab.pq.rs'
```

On commence par créer la classe IPv4 suivante.

Code Python

```python
class IPv4(object):
    def __init__(self, adresse:str):
        """ Constructeur de la classe de calcul sur une adresse IPv4
        dont la notation décimale pointée est passée en paramètre. """
        self.adresse = adresse
    def octets(self)->list[int]:
        """ Découpe l'adresse décimale pointée en la liste des 4 entiers
        correspondants.
        >>> ip01 = IPv4('192.168.1.100')
        >>> ip01.octets()
        [192, 168, 1, 100]        """
        return [int(i) for i in self.adresse.split(".")]
```

Pour mémoire, l'opérateur & entre deux entiers effectue le ET logique bit à bit de la représentation binaire de ces entiers et renvoie l'entier correspondant à la représentation binaire ainsi obtenue. Exemple :

```
>>> 192 & 255
192
```

9. Compléter les lignes 11 et 12 du code de la méthode `masquer` de la classe `IPv4`, donné ci-dessous. La méthode doit correspondre au docstring. (/1)

Code Python

```python
1  def masquer(self, masque: str)->str:
2      """ Détermine le préfixe masqué de l'adresse,
3      le masque (décimal pointé) étant passé en paramètre.
4      >>> ip01 = IPv4('192.168.1.100')
5      >>> ip01.masquer('255.192.0.0')
6      '192.128.0.0'        """
7      tmp = [ ]
8      ip = self.octets()
9      crible = IPv4(masque).octets()
10     for i in range(4):
11         tmp.append(...... & ......)
12     return ".".join(.........)
```

```
Ligne 11: tmp.append(ip[i] & crible[i])
Ligne 12: ".".join([str(k) for k in tmp])
```

Code Python

```
1  def adresse_suivante(self, adresse_max:str)->str:
2      """ Détermine l'adresse décimale pointée suivant immédiatement
3      l'adresse courante, sous réserve d'existence d'une adresse disponible
4      >>> add = IPv4('192.168.1.100')
5      >>> add.adresse_suivante('192.168.1.254')
6      '192.168.1.101'
7      >>> add = IPv4('192.168.1.255')
8      >>> add.adresse_suivante('192.168.255.254')
9      '192.168.2.0'        """
10     assert self.adresse < adresse_max
11     liste_courante = self.octets()
12     liste_suivante = list()
13     retenue = 1
14     for index in range(4):
15         somme = liste_courante[3 - index] + retenue
16         valeur, retenue = ......, ......
17         liste_suivante =......
18     return '.'.join(liste_suivante)
```

10. Compléter les lignes 16 et 17 du code de la méthode `adresse_suivante` de la classe `IPv4`, donné ci-dessous. La méthode doit correspondre au docstring. (/1)

```
Ligne 16: valeur, retenue = somme%256, somme//256
Ligne 17: liste_suivante = [str(valeur)] + liste_suivante
```

Exercice 2 ▶ /8 points

PARTIE A

Dans cette partie, on pourra utiliser les clauses du langage SQL pour :

- construire des requêtes d'interrogation à l'aide de `SELECT`, `FROM`, `WHERE` (avec les opérateurs logiques `AND`, `OR`), `JOIN... ON`;
- construire des requêtes d'insertion et de mise à jour à l'aide de `UPDATE`, `INSERT`, `DELETE`;
- affiner les recherches à l'aide de `DISTINCT`, `ORDER BY`.

La compagnie aérienne AirInfo souhaite utiliser un système de gestion de bases de données relationnelles afin de l'aider à gérer les informations dont elle dispose sur les aéroports desservis, les vols proposés, les passagers et les réservations effectuées par ces passagers.

Elle crée donc la base de données `compagnie_aerienne`, constituée de quatre relations. Le schéma relationnel de cette base de données est le suivant :

```
aeroport(id_aeroport : TEXT, ville : TEXT, pays : TEXT)
vol(id_vol : TEXT, aeroport_dep : TEXT, aeroport_arr : TEXT, dist : INT)
passager(id_passager : INT, nom : TEXT, ville : TEXT, d_totale : INT)
reservation(id_vol : TEXT, id_passager : INT)
```

Une clé primaire de la relation vol est l'attribut `id_vol`. Les autres clés primaires et étrangères ne sont pas précisées.

Les quatre tables de l'**annexe 2** constituent, en l'état, la base de données `compagnie_aerienne` complète.

Les valeurs des champs `distance` et `d_totale` intervenant respectivement dans les tables `vol` et `passager` sont exprimées en milliers de kilomètres.

ANNEXE 2

11. Expliquer pourquoi l'attribut `id_vol` ne peut pas être une clé primaire de la table `reservation`. (/0.5)

> L'attribut `id_vol` ne peut pas être une clé primaire de la table `reservation` car il peut y avoir plusieurs réservations pour un même vol, comme le montre la table `reservation` où le vol AI0006 est réservé par deux passagers différents (`id_passager` 1 et 2). Par conséquent, l'attribut `id_vol` ne permet pas d'identifier de manière unique chaque enregistrement de la table `reservation`.

12. Proposer une clé primaire pour la table `reservation`. (/0.5)

> Une clé primaire pour la table `reservation` pourrait être la combinaison des attributs `id_vol` et `id_passager`. En effet, cette combinaison permettrait d'identifier de manière unique chaque enregistrement de la table, car un même vol peut être réservé par plusieurs passagers, mais un même passager ne peut pas réserver le même vol plus d'une fois.

13. Expliquer le rôle d'une clé étrangère dans une relation. (/0.5)

> Une clé étrangère dans une relation est un attribut ou un ensemble d'attributs qui fait référence à la clé primaire d'une autre relation. Son rôle est de créer un lien entre les deux relations, permettant ainsi d'assurer l'intégrité référentielle des données. En utilisant des clés étrangères, on peut garantir que les données dans une table sont cohérentes avec les données dans une autre table, et cela facilite également les opérations de jointure entre les tables lors de l'interrogation de la base de données.

14. Écrire le résultat renvoyé par la requête SQL suivante : (/0.5)

```sql
SELECT id_vol
FROM vol
WHERE aeroport_arr = 'CDG'
```

Le résultat renvoyé par la requête SQL serait une liste des `id_vol` des vols dont l'aéroport d'arrivée est `'CDG'`. En se basant sur la table `vol`, les vols correspondants seraient AI0015, AI0258, AI0292.

15. Écrire une requête SQL qui donne les noms des villes qui sont destination d'un vol au départ de l'aéroport CDG. (/0.5)

```sql
SELECT aeroport.ville
FROM aeroport
JOIN vol ON aeroport.id_aeroport = vol.aeroport_arr
WHERE vol.aeroport_dep = 'CDG';
```

La compagnie AirInfo envisage de récompenser la fidélité de ses usagers en tenant compte de la distance totale qu'ils parcourent sur leurs lignes.

Ainsi, à chaque nouvelle réservation, la table passager doit être mise à jour : si l'usager avait déjà parcouru une distance totale $d_{\texttt{totale}}$, puis réserve un vol d'une distance $d_{\texttt{vol}}$, l'attribut `d_totale` est modifié en prenant pour nouvelle valeur $d_{\texttt{totale}} + d_{\texttt{vol}}$.

16. La passagère d'identifiant 5 a déjà parcouru 10 milliers de kilomètres. Elle réserve un vol de Washington (IAD) à Paris (CDG), d'une distance de 6 milliers de kilomètres.

    Écrire la requête permettant de mettre à jour la table passager. (/0.5)

```sql
UPDATE passager
SET d_totale = d_totale + 6
WHERE id_passager = 5;
```

La compagnie aérienne offre maintenant la possibilité de relier Paris CDG à Montréal YUL (Canada) par un vol de 6 milliers de kilomètres. Afin de prendre en compte ce nouveau trajet dans la base de données, on souhaite l'ajouter dans la table `vol` dont la clé primaire est `id_vol`. On propose, de manière incorrecte, la requête d'insertion ci-après :

```sql
INSERT INTO vol VALUES ('AI0256', 'CDG', 'YUL', 6);
```

17. Proposer, en justifiant, une correction relative à l'erreur commise. (/1)

La requête d'insertion proposée est incorrecte car elle utilise une valeur d'identifiant de vol ('AI0256') qui existe déjà dans la table `vol`. En effet, l'identifiant 'AI0256' est déjà utilisé pour le vol de CDG à SYD. Pour corriger cette erreur, il faut choisir un nouvel identifiant de vol qui n'existe pas encore dans la table, par exemple 'AI0257'. La requête corrigée serait donc :

```sql
INSERT INTO vol VALUES ('AI0257', 'CDG', 'YUL', 6);
```

PARTIE B

Dans cette partie, on considère les villes suivantes : Washington(W) ; Paris(P) ; Berlin(B) ; Tokyo(T) ; Sydney(S).

Les distances des vols entre ces villes, exprimées en milliers de kilomètres, sont les suivantes :

- Washington – Paris : 6 ;
- Paris – Tokyo : 10 ;
- Paris – Sydney : 17 ;
- Berlin – Tokyo : 9 ;
- Tokyo – Sydney : 8.

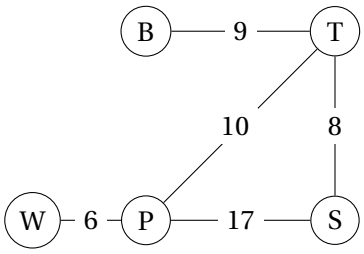


FIGURE 2 – Graphe du réseau aérien

La compagnie AirInfo propose certains de ces vols. Son réseau aérien est représenté par le graphe pondéré non orienté de la figure suivante dans lequel :

- chaque ville est représentée par un sommet ;
- chaque vol direct entre deux villes est représenté par une arête.

Le nombre indiqué sur chaque arête est appelé poids : il représente la distance entre deux villes. Cette distance est la même dans un sens ou dans l'autre.

On choisit de représenter ce graphe en Python à l'aide d'un dictionnaire dans lequel chaque clé est un sommet du graphe et la valeur associée à cette clé est elle-même un dictionnaire, dont les clés sont les villes voisines et les valeurs sont les distances correspondantes.

Le dictionnaire associé au graphe, représenté par la figure précédente, est donné ci-après.

Code Python

```python
graphe_airinfo = {
    'W': {'P': 6},
    'P': {'W': 6, 'T': 10, 'S': 17},
    'B': {'T': 9},
    'T': {'B': 9, 'P': 10, 'S': 8},
    'S': {'P': 17, 'T': 8}     }
```

18. Déterminer la valeur de l'expression `graphe_airinfo['T']['P']`. (/0.25)

    10

19. Écrire le code d'une fonction Python `vol_direct` permettant de déterminer s'il existe une liaison directe entre deux villes. Cette fonction prend en paramètres un dictionnaire `graphe` représentant le graphe, et deux clés du dictionnaire `ville1` et `ville2` représentant deux villes, et renvoie `True` si une telle liaison entre les villes `ville1` et `ville2` existe, c'est-à-dire si les deux sommets `ville1` et `ville2` sont reliés dans le graphe graphe par une arête, `False` sinon. Exemples : (/0.5)

Code Python

```python
>>> vol_direct(graphe_airinfo, 'T', 'B')
True
>>> vol_direct(graphe_airinfo, 'W', 'B')
False
```

Code Python

```python
def vol_direct(graphe, ville1, ville2):
    return ville2 in graphe[ville1].keys()
```

Afin de limiter leurs empreintes carbone, les voyageurs demandent fréquemment aux compagnies aériennes de déterminer, à partir d'une ville donnée, la liste des villes qu'il est possible d'atteindre par un vol direct tout en ne dépassant pas une certaine distance.

20. Écrire le code d'une fonction Python `liste_villes_proches` qui permet de répondre à la demande des voyageurs. Cette fonction prend en paramètres un dictionnaire `graphe` représentant le graphe, une clé du dictionnaire `ville` et un entier `d_max` représentant une distance et renvoie la liste des villes reliées à `ville` par une arête dans `graphe` dont le poids est au plus égal à `d_max`. Exemples : (/0.75)

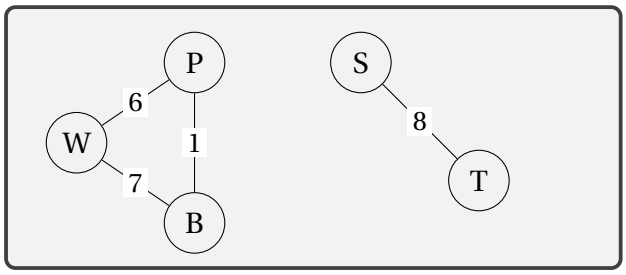
Code Python
```
>>> liste_villes_proches(graphe_airinfo, 'T', 7)
[ ]
>>> liste_villes_proches(graphe_airinfo, 'T', 9)
['B', 'S']
```

Code Python
```python
def liste_villes_proches(graphe, ville, d_max):
    liste = [ ]
    for ville_voisine, distance in graphe[ville].items():
        if distance <= d_max:
            liste.append(ville_voisine)
    return liste
```

La compagnie aérienne Droidevant, concurrente de la compagnie AirInfo, assure des liaisons différentes. Son réseau aérien peut également être représenté par un graphe dont le dictionnaire correspondant en Python est donné ci-après.

21. Dessiner, en indiquant le poids sur chaque arête, le graphe représentant le réseau aérien de la compagnie Droidevant. (/0.5)

Code Python
```python
graphe_droidevant = {
    'W': {'P': 6, 'B': 7},
    'P': {'W': 6, 'B': 1},
    'B': {'W': 7, 'P': 1},
    'T': {'S': 8},
    'S': {'T': 8} }
```



On peut adapter un algorithme de parcours de graphe pour déterminer si un graphe est connexe.

On considère la fonction parcours qui permet d'explorer les villes d'un graphe accessibles à partir d'une ville donnée et qui prend en paramètres :

- `graphe` : un dictionnaire représentant le graphe ;
- `visitees` : une liste des villes déjà visitées ;
- `ville` : la ville actuelle à partir de laquelle on effectue l'exploration.

Code Python
```python
def parcours(graphe, visitees, ville):
    """Parcours d'un graphe à partir d'une ville non visitée,
    en ayant déjà visité un certain nombre de villes."""
    visitees.append(ville)              # Marque la ville comme visitée
    for voisine in graphe[ville].keys():        # Parcourt les voisines de la ville
        if voisine not in visitees:
            # Explore depuis les voisines non visitées
            parcours(graphe, visitees, voisine)
```

22. Expliquer en quoi la fonction `parcours` est une fonction récursive. (/0.25)

> La fonction `parcours` est une fonction récursive car elle s'appelle elle-même.

23. Déterminer le contenu des variables `visitees1` et `visitees2` après l'exécution des lignes de code données ci-après. (/0.25)

```python
visitees1 = [ ]
parcours(graphe_airinfo, visitees1, 'W')
visitees2 = [ ]
parcours(graphe_droidevant, visitees2, 'W')
```

> Après l'exécution de la fonction `parcours` sur le `graphe_airinfo` à partir de la ville `'W'`, la variable `visitees1` contiendra les villes accessibles depuis `'W'` dans ce graphe, soit `['W', 'P', 'T', 'B', 'S']`.
> Après l'exécution de la fonction `parcours` sur le `graphe_droidevant` à partir de la ville `'W'`, la variable `visitees2` contiendra les villes accessibles depuis `'W'` dans ce graphe, soit `['W', 'P', 'B']`.

24. Indiquer, parmi les propositions suivantes, laquelle correspond au type de parcours effectué par la fonction parcours (cochez le bonne réponse) : (/0.25)

- ☐ Proposition A : un parcours en largeur ;
- ☐ Proposition B : un parcours en grandeur ;
- ☐ Proposition C : un parcours en profondeur.

> Réponse : Proposition C : un parcours en profondeur (DFS).

On cherche à écrire une fonction `est_connexe` qui prend en paramètre un graphe, représenté à l'aide d'un dictionnaire de dictionnaires de voisins, et qui renvoie `True` si le graphe est connexe, `False` sinon.

On admet disposer d'une fonction `ville_arbitraire` qui renvoie un sommet arbitraire d'un graphe.

Par exemple, `ville_arbitraire(graphe_airinfo)` pourrait renvoyer `'T'` ou n'importe quel autre sommet.

On considère le code de la fonction Python incomplet suivant :

```python
1 def est_connexe(graphe):
2     depart = ville_arbitraire(graphe)
3     visitees = ...
4     parcours(graphe, visitees, depart)
5     return ...
```

25. Recopier et compléter les lignes 3 et 5 du code de la fonction `est_connexe`. On pourra, si nécessaire, ajouter de nouvelles lignes. (/0.5)

```python
def est_connexe(graphe):
    depart = ville_arbitraire(graphe)
    visitees = [ ]
    parcours(graphe, visitees, depart)
    return len(visitees) == len(graphe)
```

On considère maintenant le code de la fonction, donné ci-après, `mystere`, qui prend en paramètres :

- `graphe`, un dictionnaire représentant un graphe ;
- `ville`, une clé du dictionnaire graphe représentant une ville de départ ;
- `chemin`, une liste de clés représentant les villes déjà visitées sur le chemin parcouru depuis la ville de départ ;
- `cout`, un entier représentant une distance parcourue depuis la ville de départ ;
- `arrivee`, une clé du dictionnaire graphe représentant une ville d'arrivée.

Code Python

```python
def mystere(graphe, ville, chemin, cout, arrivee):
    # Ajoute la ville actuelle au chemin
    chemin = chemin + [ville]
    if ville == arrivee:
        # Affiche le chemin et son coût total
        print(chemin, cout)
    # Parcourt les villes voisines et leurs poids
    for voisine, poids in graphe[ville].items():
        # Vérifie que la ville n'est pas déjà visitée
        if voisine not in chemin:
            mystere(graphe, voisine, chemin, cout + poids, arrivee)
```

26. Déterminer les affichages produits lors de l'exécution de l'appel suivant : (/0.5)

Code Python

```python
mystere(graphe_airinfo, 'W', [ ], 0, 'B')
```

Les affichages produits lors de l'exécution de l'appel suivant :

Code Python

```
['W', 'P', 'T', 'B'] 25
['W', 'P', 'S', 'T', 'B'] 40
```

27. Expliquer, de manière générale, ce que réalise l'appel `mystere(graphe, ville, [ ], 0, arrivee)` pour un graphe `graphe` et deux villes `ville` et `arrivee`. (/0.25)

L'appel `mystere(graphe, ville, [ ], 0, arrivee)` réalise une recherche de tous les chemins possibles entre la ville de départ (`ville`) et la ville d'arrivée (`arrivee`) dans le graphe `graphe`.
La fonction utilise une approche récursive pour explorer les différentes routes possibles. À chaque étape, elle ajoute la ville actuelle au chemin parcouru et met à jour le coût total en ajoutant le poids de l'arête correspondante.
Lorsque la fonction atteint la ville d'arrivée, elle affiche le chemin parcouru ainsi que son coût total. En explorant toutes les voies possibles, elle permet de trouver tous les chemins entre les deux villes et leurs coûts associés.

Exercice 3 ▶ /5 points

Le jeu du baguenaudier est un jeu de casse-tête constitué d'une réglette comportant `n` cases, numérotées de `1` à `n`. Chaque case peut être soit vide, soit contenir un pion.

« Jouer » une case consiste à placer un pion dans la case (remplir) si elle est vide ou enlever un pion (vider) si elle est remplie. Initialement, toutes les cases sont vides.

Le but du jeu est de remplir toutes les cases du baguenaudier en suivant les règles suivantes :

- on ne peut jouer qu'une case à la fois ;
- chaque case ne peut contenir qu'un pion ;
- on peut toujours jouer la case `1` ;
- si le baguenaudier n'est ni vide ni rempli, on peut aussi jouer la case qui suit la première case remplie ;
- aucune autre case ne peut être jouée.

Exemple de situation avec un baguenaudier de 5 cases :

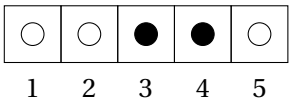


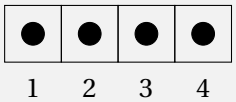
FIGURE 3 – Situation baguenaudier de 5 cases

Dans cette situation, on peut poser un pion dans la case 1 ou enlever le pion de la case 4, mais on ne peut pas jouer les cases 2, 3 et 5. En effet :

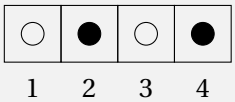
- On peut jouer la case 1 car c'est la première case du jeu.
- le baguenaudier n'est ni vide ni rempli, la première case remplie est la case 3, on peut jouer la case 4 qui suit la première case remplie.
- on ne peut pas jouer la case 2 car elle ne suit pas la première case remplie.
- on ne peut pas jouer la case 3 car elle ne suit pas la première case remplie.
- on ne peut pas jouer la case 5 car elle ne suit pas la première case remplie.

28. On considère un baguenaudier à 4 cases sur lequel les trois dernières cases sont remplies. Désigner les cases que l'on peut jouer et dessiner l'état du baguenaudier à la suite de chacun des coups possibles. (/0.5)

Case 1 :



Case 3 :



Pour modéliser le baguenaudier on utilise un tableau (de type `list`) de booléens. Une case vide est représentée par `False` et une case remplie par `True`.

L'exemple de situation du baguenaudier de 5 cases de la Figure 3 présentée plus haut sera ainsi représenté par le tableau suivant :

Code Python
```python
[False, False, True, True, False]
```

La fonction `initialiser` prend en paramètre un nombre entier `n` et elle renvoie une liste modélisant un baguenaudier vide de `n` cases. Exemple :

Code Python
```python
>>> initialiser(3)
[False, False, False]
```

29. Ecrire le code de la fonction `initialiser(n)` (/0.5)

Code Python
```python
def initialiser(n):
    return [False] * n
```

La fonction `victoire(tab)` renvoie un booléen indiquant si toutes les cases du baguenaudier sont remplies.

Code Python
```python
1 def victoire(tab):
2     for etat_case in tab:
3         if etat_case == ......:
4             return ......
5     return ......
```

30. Compléter les lignes 3, 4 et 5 du code de la fonction `victoire`. (/0.5)

Code Python
```python
def victoire(tab):
    for etat_case in tab:
        if etat_case == False:
            return False
    return True
```

La fonction `indice_premiere_case_occupee(tab)` prend en paramètre un tableau `tab` modélisant l'état du baguenaudier et renvoie l'indice de la première case remplie du baguenaudier, ou `None` si le baguenaudier est vide. Exemples :

Code Python
```python
>>> indice_premiere_case_occupee([False, False, True, True, False])
2
>>> indice_premiere_case_occupee([True, True, True, True, True])
0
>>> indice_premiere_case_occupee([False, False, False, False, False])
None
```

31. Ecrire le code de la fonction `indice_premiere_case_occupee` (/0.5)

Code Python
```python
def indice_premiere_case_occupee(tab):
    for i in range(len(tab)):
        if tab[i] == True:
            return i
    return None
```

La fonction `coup_valide(tab, case)` prend en paramètres `tab` qui est une liste modélisant l'état d'un baguenaudier, et un entier `case` qui est l'indice de la case à jouer. La fonction renvoie `True` si le coup respecte les règles et `False` si ce n'est pas le cas. L'indice de la première case du jeu est `0`. Pour que le coup soit valide, il faut aussi s'assurer que l'indice est un entier positif inférieur à la longueur de `tab`.

32. Ecrire le code de la fonction `coup_valide(tab, case)` (/1)

Code Python

```python
def coup_valide(tab, case):
if case < 0 or case >= len(tab):      # Vérification que l'indice est valide
    return False
if case == 0:                         # La case 0 est toujours jouable
    return True
# Pour toute autre case, le baguenaudier doit être ni vide ni rempli,
# et la case doit suivre immédiatement la première case remplie
i = indice_premiere_case_occupee(tab)
if i is None:          # Baguenaudier vide : seule la case 0 est autorisée
    return False
if victoire(tab):      # Baguenaudier rempli : seule la case 0 est autorisée
    return False

return case == i + 1
```

La fonction `changer_case(tab, case)` prend en paramètres un tableau `tab` modélisant l'état du baguenaudier et un indice de case nommé `case`. Si le coup désigné par `case` est valide, la fonction renvoie le tableau modélisant le baguenaudier obtenu après avoir changé l'état de la case correspondante. Si le coup joué n'est pas valide, le baguenaudier passé en paramètre est renvoyé sans modification. Exemples :

Code Python

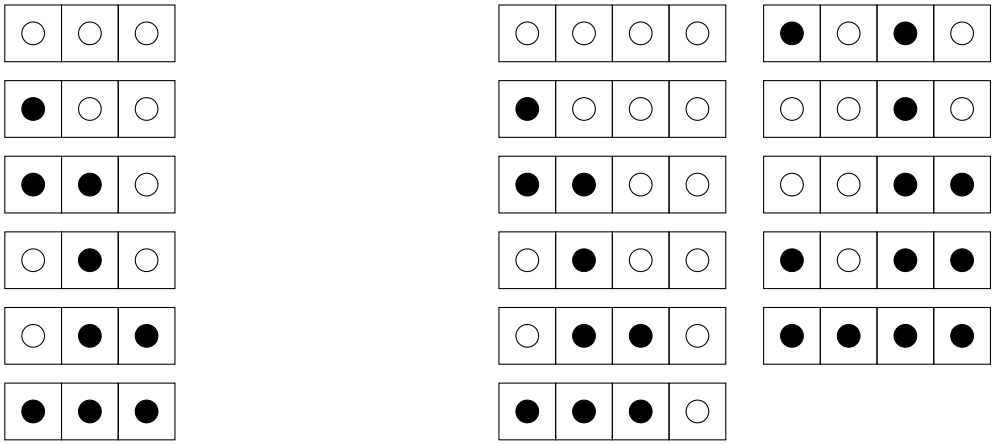
```python
>>> changer_case([False, False, False], 1)  # coup non valide
[False, False, False]
>>> changer_case([False, False, False], 0)  # coup valide
[True, False, False]
>>> changer_case([False, True, False], 2)   # coup valide
[False, True, True]
```

33. Ecrire le code de la fonction `changer_case(tab, case)` (/0.5)

Code Python

```python
def changer_case(tab, case):
    if coup_valide(tab, case):
        tab[case] = not(tab[case])
    return tab
```

La résolution d'un baguenaudier de 3 cases, en Figure 4a et 4 cases, en Figure 4b sont données ci-dessous.



(a) Résolution du baguenaudier à 3 cases

(b) Résolution du baguenaudier à 4 cases

FIGURE 4 – Quelques résolutions du baguenaudier

Il est possible d'obtenir la solution du jeu du baguenaudier, sous forme d'affichage, en utilisant des fonctions récursives nommées `vider` et `remplir`. Ces deux fonctions prennent en paramètre un entier `n` correspondant au nombre de cases du jeu. La fonction `vider` vide un baguenaudier de `n` cases initialement remplies. La fonction `remplir` remplit un baguenaudier de `n` cases initialement vides. Le code de la fonction `vider`, incomplet, est donné ci-dessous. On notera que les cases sont numérotées à partir de `1` dans la suite et que l'appel `vider(0)` ne fait rien.

Code Python

```python
1 def vider(n):
2     if n == 1:
3         print('Vider case 1')
4     elif n > 1:
5         vider(n-2)
6         print(......)
7         remplir(n-2)
8         vider(n-1)
```

34. Recopier et compléter la ligne 6 du code de la fonction `vider` pour qu'elle affiche le texte `'Vider case'` suivi de la valeur de l'entier `n`. (/0.5)

Code Python

```python
print('Vider case', n)
```

35. En supposant que l'appel `remplir(1)` affiche `'Remplir case 1'` et que l'appel `remplir(0)` ne fait rien, donner les affichages, dans la console, produits par l'exécution de `vider(3)`. (/0.5)

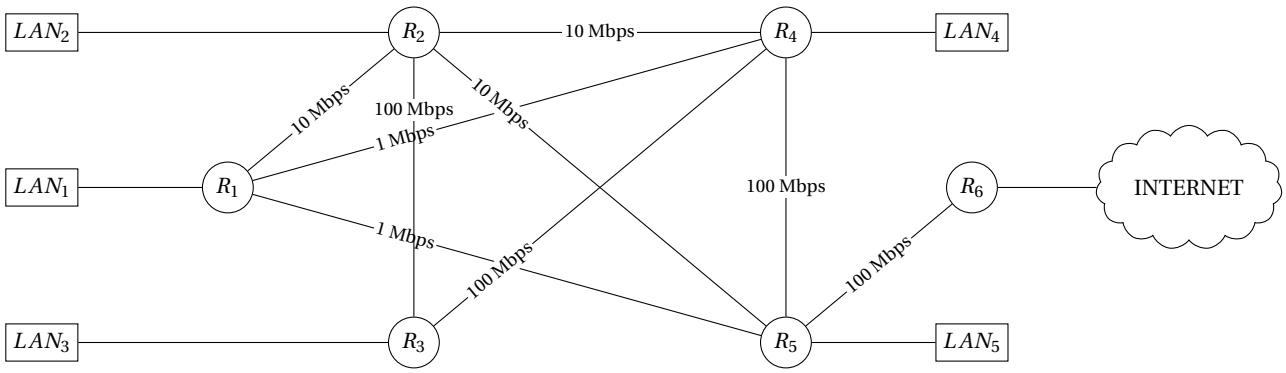
```
Vider case 1
Vider case 3
Remplir case 1
Vider case 2
Vider case 1
```

36. En vous inspirant de la fonction `vider`, de la résolution donnée à la question précédente pour `n = 3` et des Figures 4a et 4b, écrire le code de la fonction récursive `remplir(n)`. (/0.5)

Code Python

```python
def remplir(n):
    if n == 1:
        print('Remplir case 1')
    elif n > 1:
        vider(n-2)
        print('Remplir case', n)
        remplir(n-2)
        vider(n-1)
```

ANNEXE 1



ANNEXE 2

aeroport		
id_aeroport	ville	pays
CDG	Paris	France
IAD	Washington	USA
QPP	Berlin	Allemagne
NRT	Tokyo	Japon
SYD	Sydney	Australie
YUL	Montréal	Canada

passager			
id_passager	nom	ville	d_totale
1	Dupont	Paris	6
2	Smith	Wash.	6
3	Brown	Berlin	9
4	Yamada	Tokyo	17
5	Williams	Sydney	10

vol			
id_vol	aeroport_dep	aeroport_arr	dist
AI0006	CDG	IAD	6
AI0015	IAD	CDG	6
AI0256	CDG	SYD	17
AI0258	SYD	CDG	17
AI0276	CDG	NRT	10
AI0292	NRT	CDG	10
AI1280	NRT	QPP	9
AI1681	QPP	NRT	9
AI1785	NRT	SYD	8
AI1845	SYD	NRT	8

reservation	
id_vol	id_passager
AI0006	1
AI0006	2
AI0256	4
AI0276	5
AI1681	3